WHITE PAPER

mmRISC-2
A 64-bit RISC-V CPU
Designed Entirely by Claude Code

From the specification, RTL and verification to Linux on an Arty A7-100T

Munetomo Maruyama (professional name; legal name: Takanaga Yamazaki)

October 20, 2026 (covers mmRISC-2 Rev.02, the final version)

https://github.com/munetomo-maruyama/mmRISC-2

Abstract

This paper describes **mmRISC-2**, a project in which Anthropic's AI coding agent **Claude Code** designed a 64-bit RISC-V CPU (RV64GC) that boots Linux on a Digilent Arty A7-100T FPGA board. Claude Code wrote the specifications, the RTL, the testbenches and the verification, the scripts and the documents in two languages. The author set the goals, chose among the proposals, and ran Vivado and the board.

Linux booted on the home-made CPU from the SD card to the BusyBox shell **12 days** after the start, and **30 days** after the start the project reached Rev.01: an L2 cache, a pipelined FPU, Zba / Zbb / Zbs / Zbc / Zicond / Zicboz, 3.025 CoreMark/MHz, and a 120-minute stress test passed. **Rev.02**, the final version **38 days** after the start, added larger TLBs (up to 20 % faster on Linux loads), 3.034 CoreMark/MHz and a software environment: **bare-metal C programs in the Eclipse IDE**, debugged on the board over JTAG / cJTAG, and **Debian 13 booting on the board**, where gcc compiles programs on mmRISC-2 itself. The paper covers how the work was done, the specification, the history, the rework and how verification improved, an evaluation of AI-driven design, and the role left to the human.

Contents

1. Introduction
2. How the work was done
3. Specification (Rev.02)
4. History of the development
5. Rework and verification
6. Evaluation of design by Claude Code
7. Summary and next steps
Appendix A. About the author
Appendix B. References

mmRISC-2 Rev.02 in numbers

12 days
From the start to Linux on the home-made CPU
(2026-09-13 → 09-25)

38 days
To Rev.02, the final version, 212 commits
(Rev.01 in 30 days; → 2026-10-20)

22,100 lines
RTL (SystemVerilog, 46 files)
spec, verification, docs: Claude Code too

Debian 13
Linux 7.2 from the SD card, BusyBox or Debian;
gcc compiles on the board itself

3.034 /MHz
CoreMark (fastest options, Zba/Zbb)
2.794 with -O2, DMIPS 1.545

554
Bug injections, all detected
(do the tests find broken RTL?)

1. Introduction

1.1 Goal

- Build a 64-bit RISC-V CPU (**RV64GC**) and run **Linux** on it on the FPGA board **Arty A7-100T**.
- Make **only the hardware of the CPU core**. The peripherals and the SoC come from LiteX; OpenSBI, Linux and BusyBox are used as they are.
- **Leave the design to Claude Code**. Claude Code writes the specifications, the RTL, the testbenches, the scripts and the documents; the author sets the goals, chooses among the proposals, and handles Vivado and the board.

The question: how far can an AI design a CPU, and what should the human do?

Setting the constraint "make only the CPU, reuse everything else" at the very beginning was one of the main reasons the project reached Linux so quickly.

2. How the work was done

2.1 The first prompts and the choice of a base

The project started with the following request (translated from Japanese).

① The first request (early morning, 2026-09-13)

> *"I want to design my own 64-bit RISC-V CPU core in Verilog HDL and run 64-bit Linux on an FPGA board. I will design only the hardware of the CPU core and reuse the existing peripherals and Linux as they are. Should I use LiteX, or vivado-risc-v, or is there another good base? The FPGA board is the Digilent Arty A7."*

② 09-13 04:40, the instruction to start

> *"First, build a 64-bit RISC-V system on the Digilent Arty A7 board based on standard LiteX, and run Linux on it. Please go step by step."*

③ 13:07 the same day, Linux booted on the standard Rocket → 14:31, on to replacing the CPU

> *"I want to move on to swapping the CPU core. First, for information: where is the boundary of the pure CPU core to be replaced in the current system?"*

The bases were compared first, and **Linux was run on the standard configuration to make a reference** before only the CPU was replaced. At that point there was not yet a single line of CPU RTL. Table 1 summarizes Claude Code's comparison of the bases.

Table 1 Comparison of the bases (a summary of Claude Code's answer)

Base	Ease of replacing the CPU	Arty A7 support	Linux boot record	Decision
LiteX	◎ Has a mechanism for swapping CPUs; existing CPUs serve as examples	◎ Official (DDR3, Ethernet, SD)	◎ OpenSBI and device tree included	**Chosen**
vivado-risc-v	△ Built around Rocket Chip	○ Less freedom in the SoC	○ Rocket + Linux	No
CVA6 SoC	△ SoC integration is not general	△ Little support	○ 64-bit Linux	No
Chipyard	△ Chisel / Scala centered	—	○	No

The other obstacle was Vivado. Vivado runs only on x86 and cannot be installed on Ubuntu on an Apple Silicon Mac. Vivado 2025.1 did run on Windows 11 for Arm under Parallels, so the work folder was shared and only synthesis and programming were left to the author.

2.2 Division of work

Table 2 What the author and Claude Code did

The author	Claude Code
Set the goals, constraints and priorities	Wrote the specifications and recorded the design decisions (82)
Had options proposed and chose among them	Wrote all of the RTL (22,100 lines of SystemVerilog)
Provided the specification PDFs (Debug Spec, privileged spec and others)	Built and ran the testbenches, reference models and bug injection
Synthesized with Vivado and programmed the board	Integrated the CPU into LiteX; built OpenSBI, the device tree and the Linux side
Made the SD card, ran the board and passed back logs and numbers	Read the timing reports and board numbers, found causes and proposed next steps
Tried the software environments (Eclipse, Debian) on the board	Made the bare-metal kit, the Eclipse projects and the Debian root file system, and wrote how to reproduce them
Looked at the results and pointed out what was wrong	Wrote the documents (EN / JA), and committed and pushed to GitHub on its own

Claude Code wrote the RTL, the testbenches and the documents. The author's input was about 400 instructions in Japanese, plus the results from the board.

2.3 Development environment

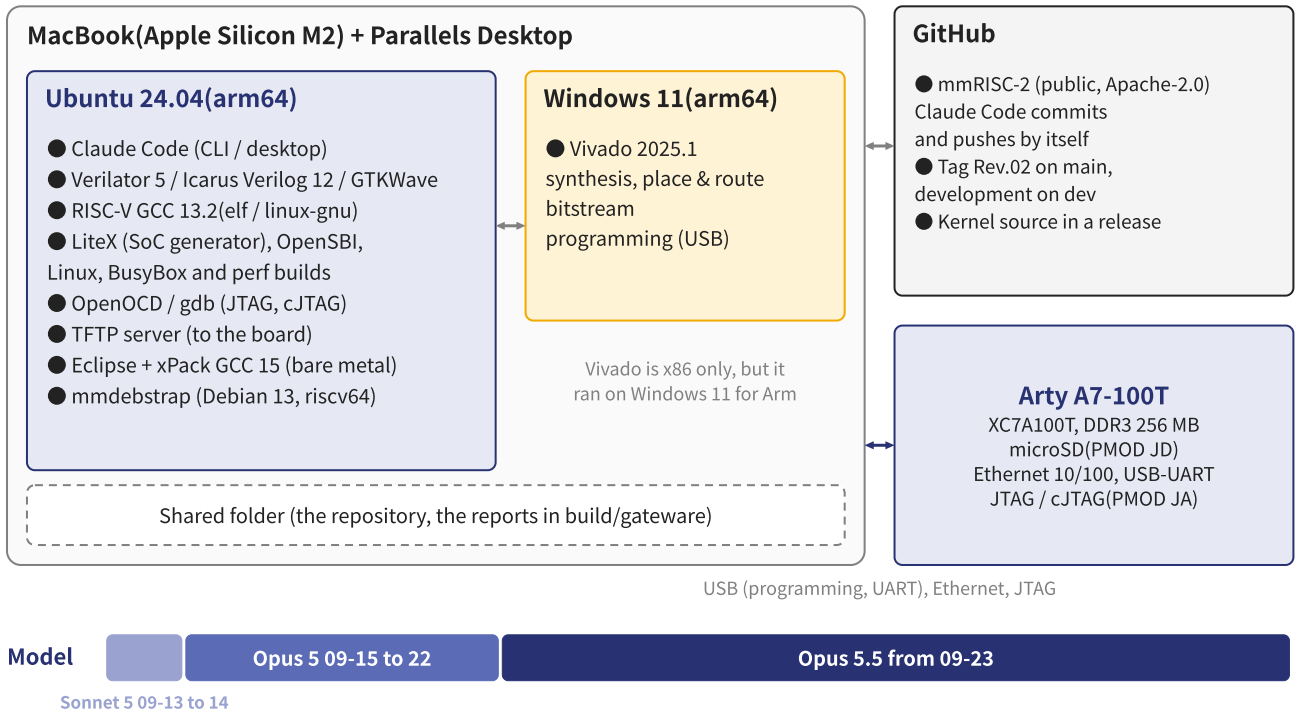


Figure 1 Development environment, tools and the models used

On Ubuntu (arm64), Claude Code ran the simulations, the software builds, and the documents and repository; only Vivado was run by the author on the Windows side. Claude Code read the Vivado reports through the shared folder. The model was Claude Sonnet 5 for the first two days (setting up LiteX with Rocket), Opus 5 from 09-15 when the work on the home-made CPU began, and Opus 5.5 from 09-23.

2.4 The design cycle

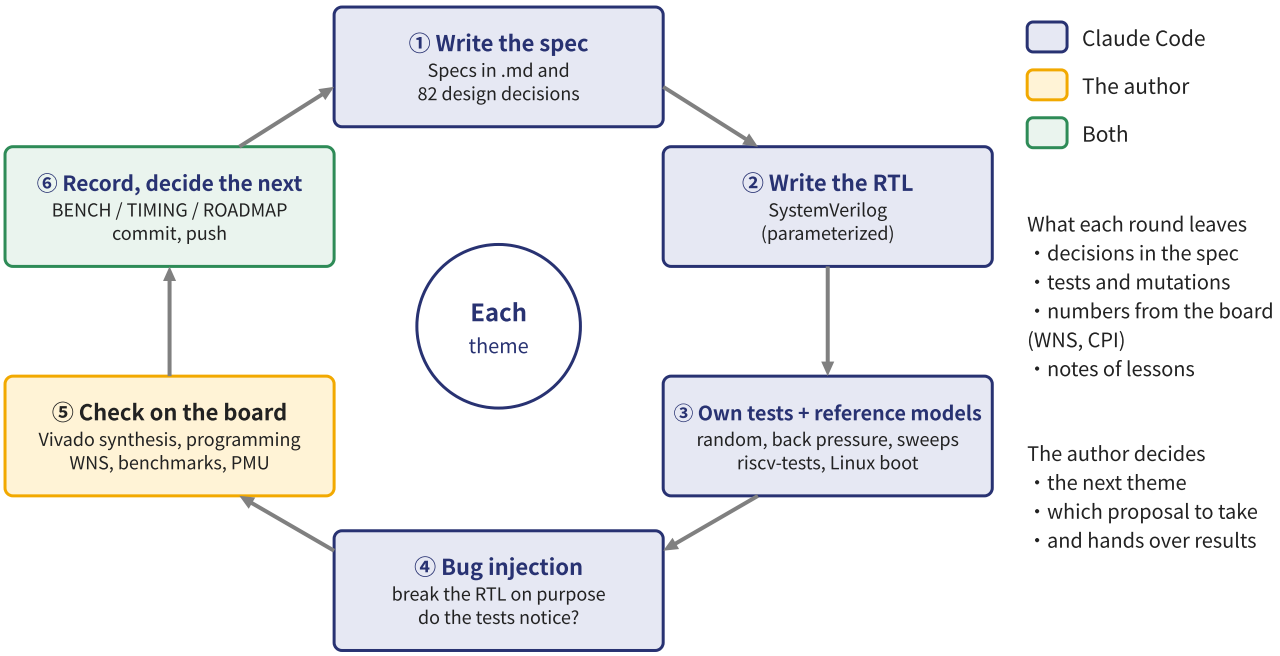


Figure 2 How Claude Code designed: a cycle run for each theme

For each theme (for example, adding the L2, or not waiting for store misses) the same cycle was run: specification → RTL → own tests and reference models → bug injection to check how much the tests catch → check on the board by the author → record the results and decide the next theme. The board's results become the input of the next cycle.

2.5 Records and scale

- **Write the specification first and keep it in step with the implementation** (82 design decisions in CPU_CORE_SPEC, each with its reasons).
- **Record the board's results chapter by chapter** (BRINGUP 20 rounds, TIMING 47 sections, BENCH 28 sections).
- **Manage themes in a ROADMAP** and mark them when done.
- **Documents in two languages** (XXXX.md / XXXX_J.md).
- **Keep lessons in Claude Code's memory notes** (e.g. make's timestamp check does not work on the shared folder; read every line of a bug-injection log).
- Even when the conversation is compacted, work continues from the documents and the notes.

Item	Value
Period	2026-09-13 to 10-20 (38 days)
Commits	212 (git from 09-19)
Author's instructions	about 400
RTL	46 files, 22,100 lines
Verification	tests and scripts, about 29,500 lines
Documents	EN 10,600 + JA 10,300 lines
Design decisions	82
Bug injections	554, all detected
API calls	about 7,100
Input tokens	about 3.3 billion (98 % cache)
Output tokens	about 8.3 million (incl. thinking)
Plan	Claude Pro

The token counts are estimates summed from Claude Code's conversation logs. They do not include the session of the first request or the compaction of conversations. Most of the input is the conversation history read again from the cache at every call.

3. Specification (Rev.02)

3.1 CPU core

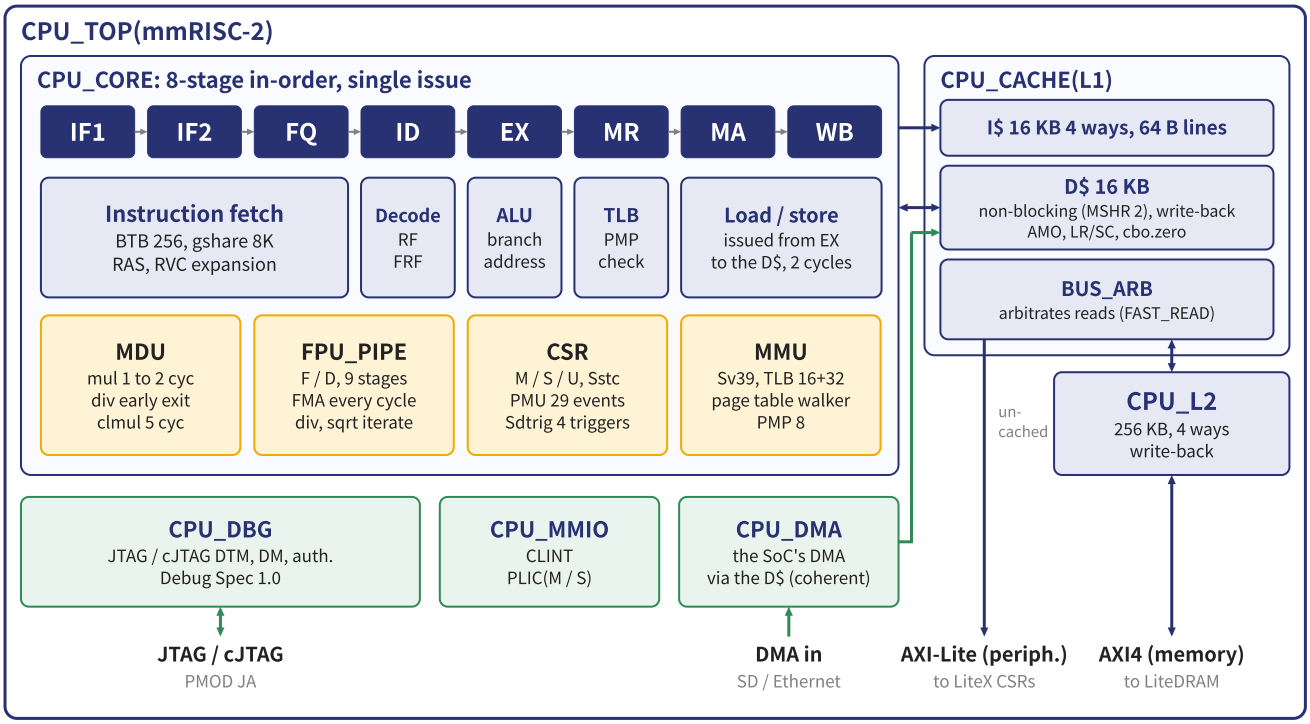


Figure 3 Block diagram of the CPU

Table 3 Main features of the CPU

Item	Specification
ISA	RV64IMAFDC + Zba / Zbb / Zbs / Zbc / Zicond / Zicboz, Zicsr, Zifencei, Zicntr, Zihpm, M / S / U
Pipeline	8-stage in-order (IF1 IF2 FQ ID EX MR MA WB), forwarding from EX, MR and MA
Branch prediction	BTB 256, gshare (PHT 8192), return address stack 8; branches on a load's value resolved in MR
Multiply / divide	Multiply in 1 to 2 cycles, divide with early exit, carry-less multiply in 5 cycles
Floating point	F / D, a 9-stage pipeline issuing one operation (FMA) a cycle; divide and square root iterate
MMU and protection	Sv39, ITLB of 16 and DTLB of 32 entries, hardware page table walker, PMP 8
Caches	L1 I$ / D$ 16 KB each (4 ways, 64 B); the D$ is non-blocking and write-back; L2 256 KB
Interrupts, privilege	CLINT, PLIC (M / S), Sstc, privileged spec 1.12
Debug	JTAG / cJTAG (Debug Spec 1.0), 4 triggers (Sdtrig), authentication
Performance counters	4 counters, 29 events, Sscofpmf, Smcntrpmf (usable from Linux's perf)
FPGA	Arty A7-100T, 50 MHz, WNS +0.140 ns, LUT 79 %, slices 93 %, BRAM 108.5 / 135

Loads and stores are issued to the D$ from EX; a hit takes 2 cycles and one access flows every cycle. The SoC's DMA also goes through the D$, so the cache coherence Linux expects is kept in hardware. The external interfaces are AXI4 for memory, AXI-Lite for peripherals, a DMA input and JTAG / cJTAG. On top of what Linux needs, extensions for speed (Zba / Zbb / Zbc, Zicboz) and a PMU for measurement were added later.

3.2 Linux system

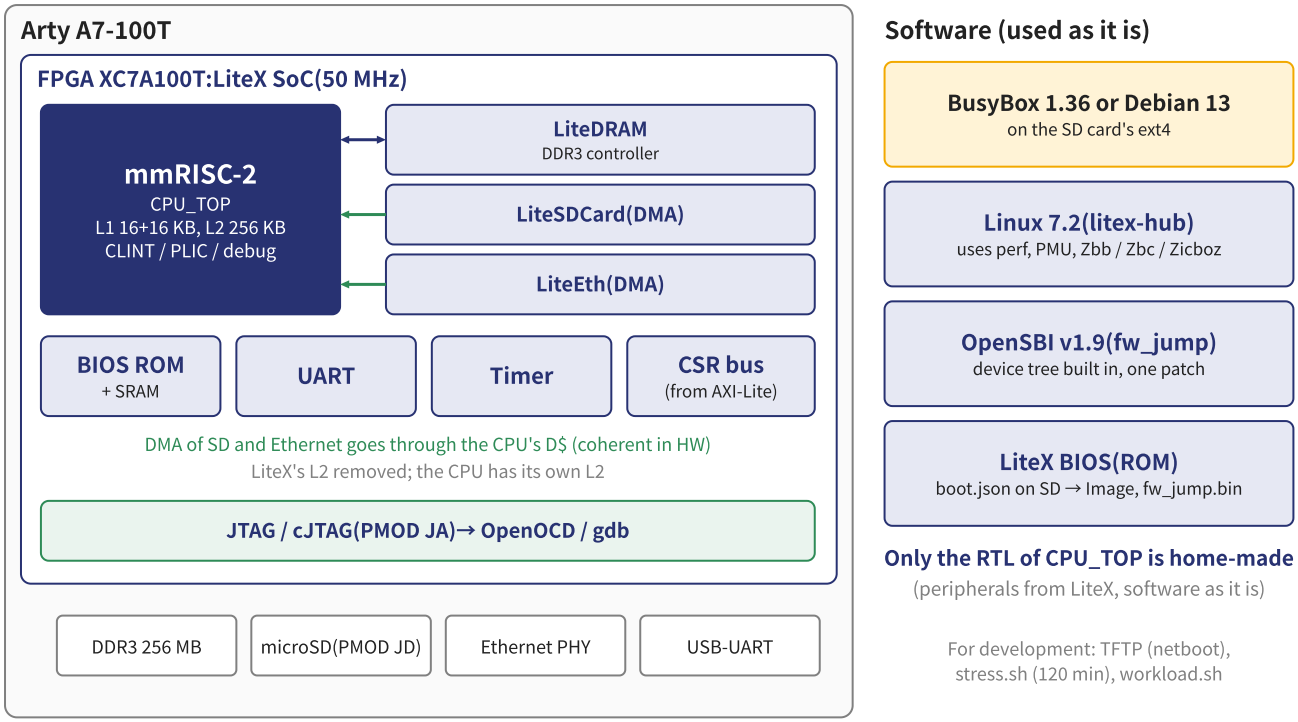


Figure 4 The Linux system

The CPU of the SoC generated by LiteX was replaced with mmRISC-2. At first the memory map matched the Rocket configuration byte for byte, and only the CPU node of the device tree was rewritten. Later LiteX's L2 was removed in favour of the CPU's own L2, the DMA was routed through the D$, and Ethernet was added. The software boots in the order LiteX BIOS → OpenSBI v1.9 → Linux 7.2 → BusyBox (or Debian 13, section 3.3).

3.3 Software development: Eclipse and Debian

Before Rev.02, two ways of writing software for mmRISC-2 were added, each with a procedure written for someone doing it for the first time (LitexSystem/docs/ECLIPSE.md and DEBIAN.md).

Bare metal in Eclipse (cross development)

- Eclipse Embedded CDT and xPack RISC-V GCC, on Ubuntu or Windows.
- A start-up kit (crt0, linker script, printf to the UART, the timer and interrupts) and example projects; a script makes a new project from a copy.
- One click builds, loads the program into DRAM over **JTAG or cJTAG** (OpenOCD) after the LiteX BIOS has set the DRAM up, and stops at `main`.
- Step through C, breakpoints, variables, registers and memory, with the on-chip debugger of section 4.1.

Debian 13 on the board (self-hosted builds)

- The second partition of the SD card holds **Debian 13** (riscv64, sysvinit) instead of BusyBox; the kernel, OpenSBI and the bitstream stay as they are.
- **gcc 14, make, gdb 16, git** on the board, and `apt` for more of Debian.
- One script makes the root file system on the host without root (mmdebstrap with qemu-user); another writes it to the card.
- A small C file compiles in about 20 s at 50 MHz; the login comes about 3 minutes after reset.

```
Debian GNU/Linux 13 mmrisc2 ttyLXU0

root@mmrisc2:~# time gcc -O2 -o t t.c && ./t
real    0m20.712s
build on mmRISC-2
```

Debian's gcc, binutils and glibc, built by others for any RV64GC machine, run on mmRISC-2 unchanged: a broad check of the user ISA by code the project did not write. Claude Code built both environments too. It generated the Eclipse project files (including the identifiers Eclipse derives from tool names), found a chain of signature checks that gets Debian 13's keys on Ubuntu, and fixed what the board showed (a missing `fsck.ext4`, and a clock stepped twice to the year 2083).

4. History of the development

4.1 What came first (09-15 to 09-20)

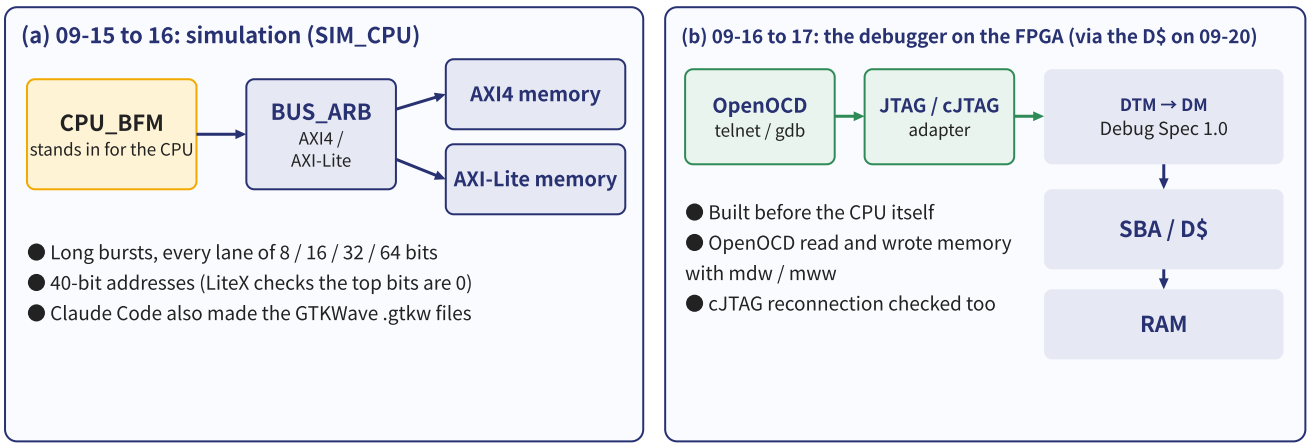


Figure 5 The bus, memory and debugger paths, checked before the CPU

Before the CPU itself, **the paths of the bus, the memory and the debugger** were checked, so that when the CPU did not work it would be clear how much was already right. The debugger was implemented before the CPU from the official Debug Spec PDF given to Claude Code; on 09-17 OpenOCD read and wrote memory on the FPGA through it, and cJTAG reconnection was checked as well. The same debugger later put hardware breakpoints on the Linux kernel running on the board. From then on the rule was "write the testbench first": for the caches, the reference model and the random tests came before the RTL.

4.2 Schedule

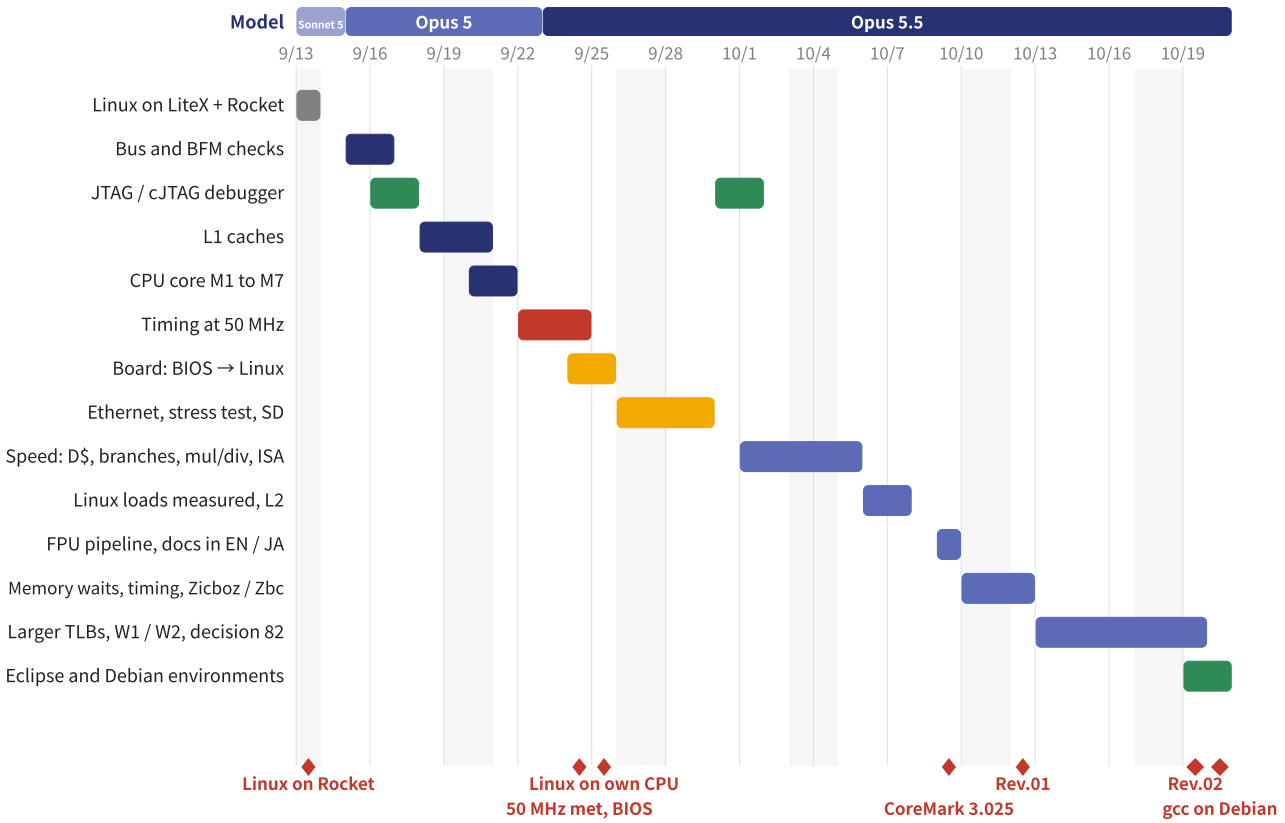


Figure 6 The actual schedule and the main events

The CPU core, from the pipeline skeleton to the integration into LiteX, took the two days of 09-20 and 21, and timing took the three days from 09-22 to 24. The BIOS came up on the board on 09-24 and Linux on 09-25. In October the work turned to performance and measurement, and from 10-10 to 12 to the memory waits and the remaining timing paths (Rev.01). From 10-13 to 19 the TLBs were enlarged, W1 and W2 were checked and decision 82 was made (the Rev.02 hardware), and on 10-19 and 20 the Eclipse and Debian environments were set up.

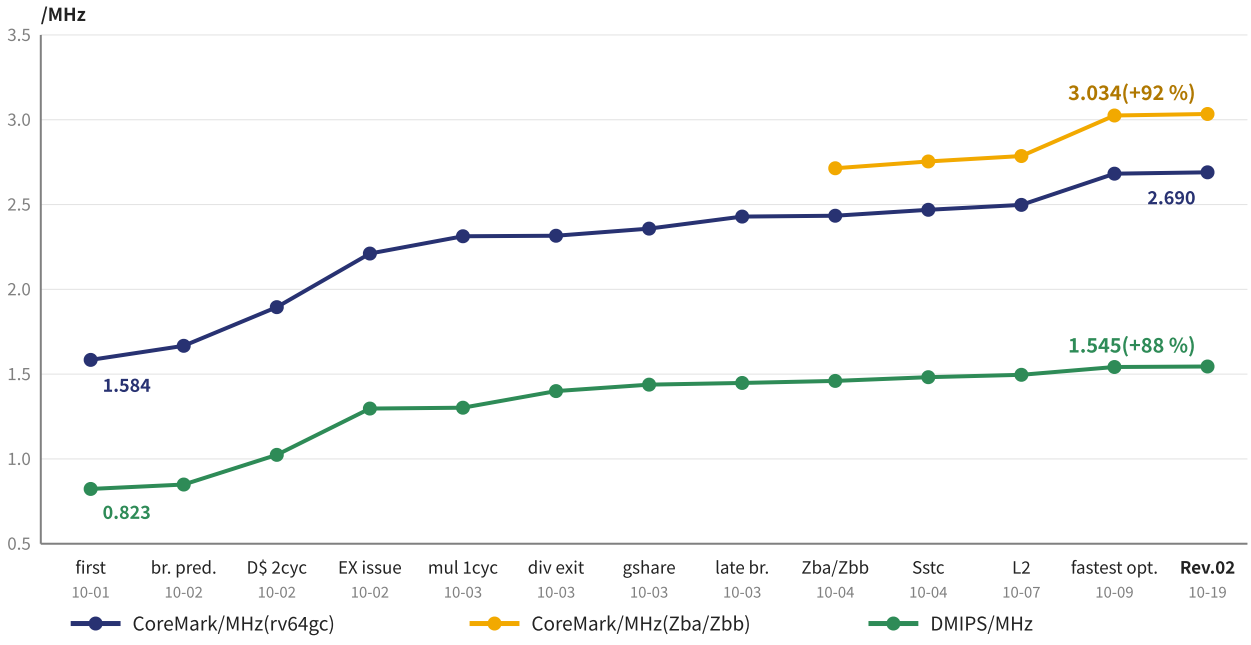
4.3 Main decisions and changes

Table 4 Main decisions and changes of the specification

Date	Decision	Reason / result
09-13	LiteX as the base; Linux on the standard Rocket first	A reference where only the CPU has to be swapped (booted the same day)
09-16	AXI4 for memory, AXI-Lite for peripherals, 40-bit addresses	Usable in other SoCs; LiteX checks that the top bits are 0
09-18	L1 like Rocket's (16 KB + 16 KB), parameterized	Verified over a sweep of 19 configurations
09-23	**Stage MR added** for 50 MHz (7 → 8 stages)	WNS −20.4 ns → 0.000 ns (09-24)
09-24	The SoC's DMA **through the D$**	Hardware keeps the coherence the Linux SD driver relies on
10-02	Loads and stores **issued from EX**	CoreMark 1.895 → 2.211 /MHz
10-06	LiteX's L2 removed; **an L2 of 256 KB inside the CPU**	Kernel-heavy loads 1.2 to 1.7 times faster
10-09	FPU **pipelined**	One operation a cycle, and fewer resources
10-11	I$ prefetch built but **switched off**	0 to 4 % slower on the board (section 5.2)
10-12	No second read on the bus; **measure first**	It would gain 0.26 %; a grant one cycle earlier (FAST_READ) instead
10-17	**ITLB 16 / DTLB 32 entries** (from 8 / 8)	Page table walks had taken 10 to 13 % of user-mode loads; awk −20.5 %, gunzip −11.4 %
10-18	Write-validate (W1) built and verified, then **not taken**; posted uncached stores (W2) not built	W1 +0.1 % on a copy in simulation, W2 under 0.1 % of tftp by measurement (section 5.2)
10-19	Decision 82: a wrong-path load no longer leaves MR waiting for the DTLB	CoreMark 3.015 → 3.034 /MHz (section 5.4)
10-20	**Eclipse** bare-metal kit; **Debian 13** with gcc on the board	Software development both ways (section 3.3); Rev.02

The author made the final choices, but Claude Code proposed most of the options and their reasoning. In the later part, "measure before building" became the rule.

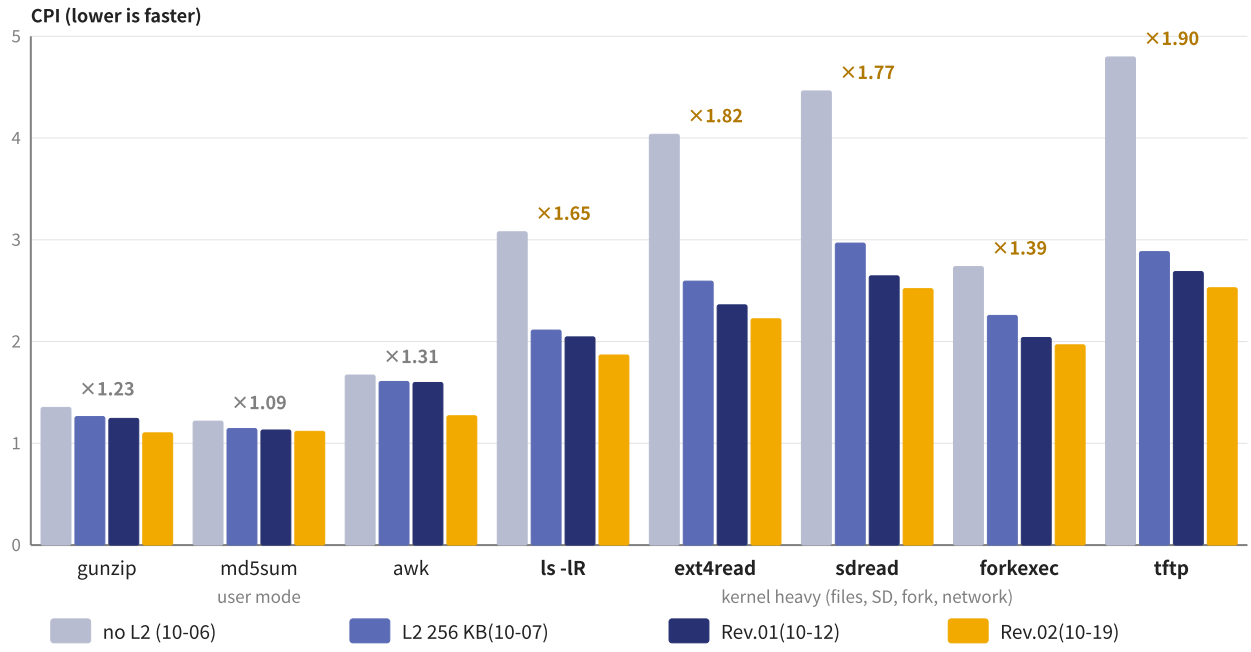
4.4 Performance



On the board (under Linux, 50 MHz). -O2 except "fastest opt." and Rev.02. BENCH.md 8, 17, 28

Figure 7 CoreMark and Dhrystone (on the board, under Linux, 50 MHz)

From 1.584 CoreMark/MHz in the first version (10-01), the steps were: D$ hits in 2 cycles, loads and stores issued from EX, 1-cycle multiply, gshare, branches on a load's value resolved in MR, Zba / Zbb, the L2 and the compiler options; Rev.02 adds decision 82. CoreMark rose by 92 % and Dhrystone by 88 %.



×: speed, no L2 → Rev.02. ls -lR also runs fewer instructions with Zbc. BENCH.md 14, 15, 25, 28

Figure 8 CPI of real Linux loads, counted by the PMU (lower is faster)

CoreMark fits in the caches, so eight real Linux loads were chosen and counted with the PMU (workload.sh). Kernel-heavy loads spent 50 to 70 % of their cycles waiting for the caches; the L2 made them 1.2 to 1.7 times faster. Not waiting for store misses, fetching before writing back a dirty victim, cbo.zero and the bus

improvements then cut another 7 to 11 %, and the larger TLBs of Rev.02 took 11 to 20 % off awk and gunzip and 3.5 to 8.7 % off the kernel loads. The breakdown by function (perf) also confirmed the faster CRC with Zbc.

4.5 Timing (WNS)

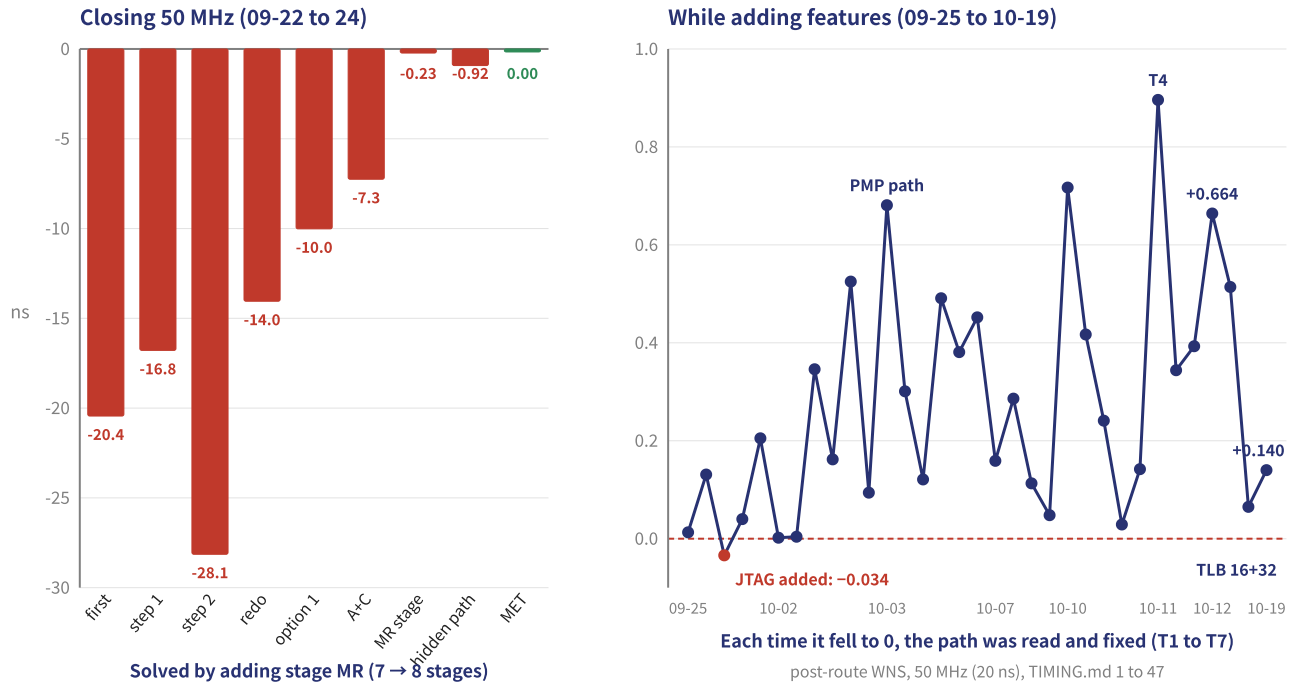


Figure 9 WNS over time (after place and route, 50 MHz)

The first synthesis gave −20.4 ns (about 25 MHz). Judging the cause to be logic depth rather than routing congestion, the paths were read and cut one by one. Step 2 made things worse and was redone, and adding stage MR finally met 50 MHz on 09-24. After that, every new feature brought WNS back near 0, and each time the worst path was read and fixed (T1 to T7). Rev.01 closed at +0.664 ns; the larger TLBs brought it down to +0.065 ns, and Rev.02 closes at +0.140 ns.

5. Rework and verification

Three cases where the AI went wrong, and what caught each of them.

5.1 Rework 1: the boot banner stopped at "__" (09-24)



On the first board run, the LiteX BIOS banner stopped part way. What got out were the 16 bytes of the UART's transmit FIFO: 2 for the newline `\n\r`, 4 for the bold escape `ESC[1m` and 10 visible characters. The CPU was running; what had stopped was the **interrupts**.

- **Cause**: uncached accesses **rounded the address to 8 bytes**. A write to the PLIC register at +4 became "the lower half with no strobes" and was lost, so no interrupt came. Beyond the 16-byte FIFO, the UART transmits from its interrupt.
- **Why it was missed**: the simulation's memory model took byte addresses, and the SoC tests tied the external interrupt to 0. **The real BIOS had never been run with interrupts.**
- **What caught it**: **SIM_BIOS**, which runs the real bios.bin with a model of the UART. The RTL before the fix was shown to stop at the same place, and then it was fixed. SIM_BIOS later became the regression test that boots Linux.

The next day a bug where instructions were not fetched again after a write of satp (the sequence that enables the MMU in S mode) was also found. Both were "paths first taken on the board".

5.2 Rework 2: a prefetch that was slower on the board (10-11)

- With a next-line prefetch for the I$, the simulated Linux boot showed CPI −1.1 %, and 48 % of the prefetches saved a miss.
- But **on the board it was 0 to 4 % slower**: real misses waited behind the bursts of prefetches that were never used.
- **Cause**: the simulation's memory answered at once, while the board's DDR3 takes tens of cycles.
- **Fix**: with a **memory latency model** (20 cycles for reads, 10 for writes), the simulation also showed +1.2 %, the same verdict as the board.

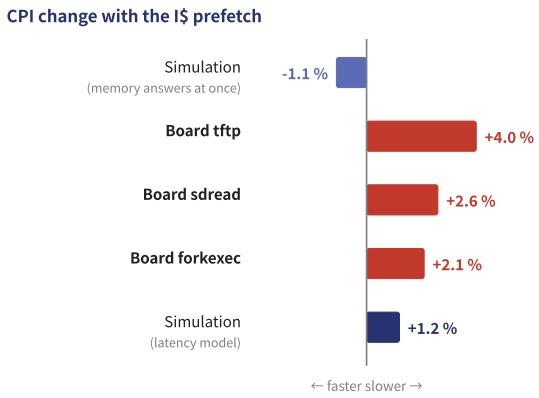


Figure 10 CPI change with the prefetch

The prefetch was switched off, and from then on the rule was **to measure before building**. For example, letting the bus take a second read was found in advance to gain only 0.26 %, and was not built. Before Rev.02, write-validate (W1), estimated at up to 6 to 8 % for SD reads, was built and verified, but with the latency model it gained 0.1 %: the copy's stores were already slow for another reason, which hid the read it was to remove. It was taken out. The AI did not question the simulation's assumption (memory answers at once) and reported "faster". **The board's numbers exposed the wrong assumption.**

5.3 Rework 3: a wrong report by the AI, and traps of the environment

A wrong report (10-12)

- When adding Zicboz, Claude Code reported and committed "59 / 59 detected" for the bug injection of the caches.
- The real count was 56 / 59: a renaming had moved the targets of three mutations.
- It had summarized only the last three lines of the log. It noticed this itself while verifying the next change, and corrected it.
- → A rule was added to its notes: "list every NOT DETECTED / NOT APPLIED line and check the count".

Traps of the environment (09-21, 10-05)

- Files written to the SD card were gone after reinserting it. macOS had mounted the card before it was handed to the VM, and wrote back its stale FAT when the card returned. Claude Code worked out the order from the FAT timestamps. → "Write the first partition from the Mac" went into the procedure and the notes.
- The shared folder keeps file times to one second, so make took freshly edited RTL as "unchanged" and tests ran on an old simulator. → Rebuilds are now decided by a hash of the sources.

The AI's mistakes show up as plausible summaries, and a rule of reading the raw logs prevents them. The traps of the environment lie on the human's side, and the AI can infer their causes from the records.

5.4 Rework 4: the last 0.3 % of CoreMark (10-19)

- The final measurements showed CoreMark **0.3 % below Rev.01's** in every build (3.015 against 3.025). The benchmark had not been run since 10-09; the core alone in simulation was not slower.
- **What found it**: a log of every TLB fill and page table walk added to SIM_BIOS. Nearly every walk in CoreMark went to **page 0**: at the end of a list a load behind a mispredicted branch, on the wrong path, missed the DTLB at address 8. The branch threw the load away but not its wait for the DTLB, a wait added on 10-11 for timing (T2).

- **Fix** (decision 82): the same flush also clears the wait. 3.034 CoreMark/MHz, and a test and a mutation that keep it.

The loss was 0.79 % of the cycles and hid behind a change for timing; it was found because the author asked for CoreMark not to go down at all, and Claude Code then traced it to two instructions.

5.5 How verification evolved

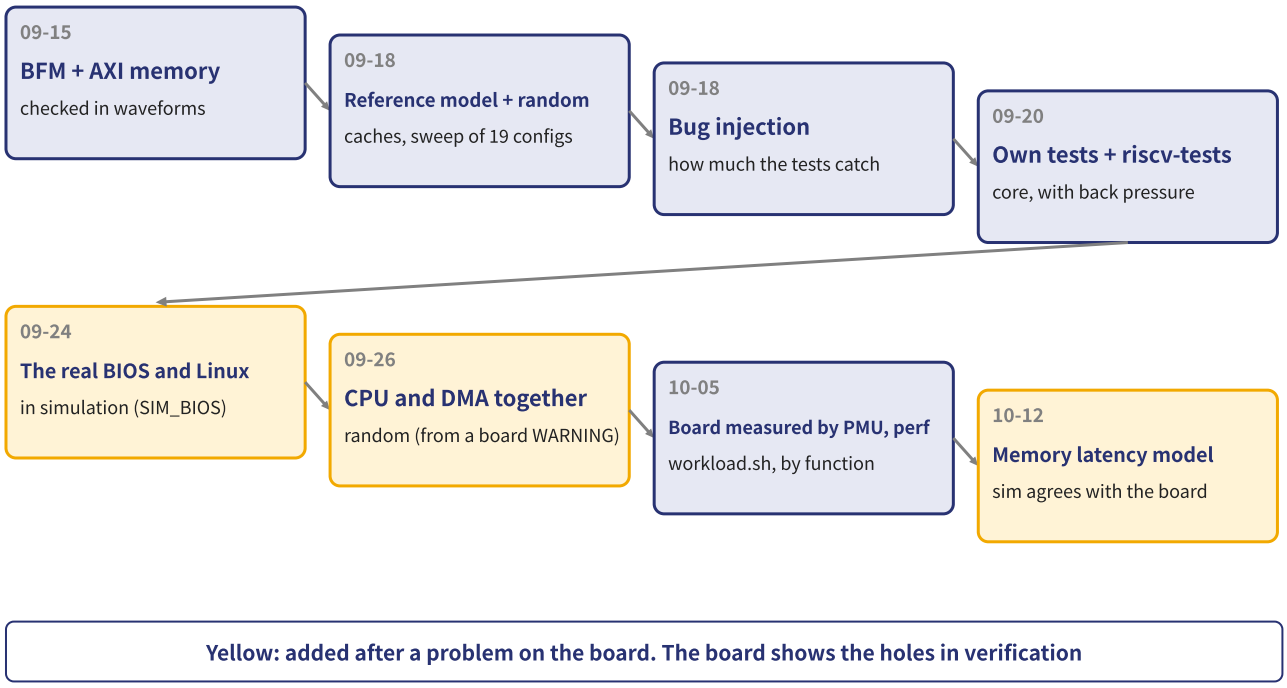


Figure 11 How verification evolved (yellow: added after a problem on the board)

Each time the board stopped, a simulation that reproduced the problem was built first, and only then was the problem fixed. The board shows where the holes in verification are.

5.6 Final verification (Rev.02)

Table 5 Final verification results

Environment	What it does	Result
SIM_CORE	34 own tests (instructions, traps, MMU, PMP, debug, PMU and more), with back pressure	All PASS
riscv-tests	Official (rv64ui/um/ua/uc/uf/ud/uzba/uzbb/uzbs/uzbc/uzicond/mi/si)	p 178 / v 154 PASS
SIM_SYS	Core + real L1 / L2 + DMA, 27 own tests + riscv-tests	All PASS
SIM_CACHE	CPU and DMA at random at the same time, 21 sections, sweep of 19 configurations	64,914 checks PASS
SIM_L2 / MMU / FPU	L2 about 1.5 million, PMP 200,000, FPU (against SoftFloat) about 1.3 million	PASS
SIM_BIOS	The real LiteX BIOS, Linux boot from an SD card model, perf	To the prompt
SIM_OCD	Co-simulation with OpenOCD (breakpoints, watchpoints)	PASS
Bug injection	CORE 340, CACHE 59, SYS 26, L2 31, MMU 21, FPU 62, DBG 15	**All 554 detected**
Board	Linux boot, Ethernet, 120-minute stress test (memory, network and SD checked at once), Debian 13 and gcc	PASS

The 4 failing riscv-tests require features not implemented (misaligned accesses in hardware, Zacas).

5.7 On the board

```
       __   _ __      _  __
      / /  (_) /____ | |/_/
     / /__/ / __/ -_)>  <
    /____/_/\__/\__/_/|_|
   Build your hardware, easily!
Memtest OK  Write 40.7MiB/s Read 41.9MiB/s
Booting from boot.json...
OpenSBI v1.9
Platform Name : mmRISC-2 on Digilent Arty A7-100T
[0.000000] Booting Linux on hartid 0
[0.000000] Linux version 7.2.0-rc2 (riscv64 gcc 13)
[0.000000] riscv: base ISA extensions acdfim
[0.159648] Memory: 239032K/262144K available
[2.608766] liteeth 12001000.ethernet eth0
[3.350734] EXT4-fs (mmcblk0p2): mounted r/w
[3.438492] Run /sbin/init as init process
udhcpc: lease of 192.168.0.9 obtained
#
```

- init 3.4 s after the kernel starts; DHCP, TFTP
- perf with function names; gdb hardware breakpoints in the running kernel
- 120-minute stress test PASS (Rev.02: mem 84, net 45, sd 502, NG 0)
- Debian 13 from the same card; gcc on the board

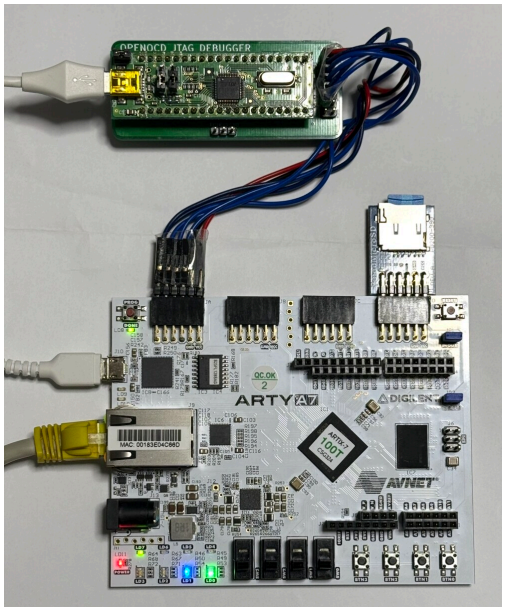


Figure 12 The Arty A7-100T and the JTAG debugger (the green board on top)

6. Evaluation of design by Claude Code

6.1 Strengths and weaknesses

Strengths (pros)	Weaknesses (cons)
Fast: Linux on a home-made CPU in 12 days, +91 % performance in 30 days	**The board needs a human**: Vivado, the board, the SD card, observation
Writes and maintains specification, RTL, verification and documents **consistently** (EN / JA)	**Carries on plausibly from a wrong assumption** ("faster" with an instantly answering memory)
Thorough verification: builds and runs reference models, random tests, sweeps and 554 bug injections itself	**Can report wrongly** (56 / 59 reported as 59 / 59)
Reads the board's numbers: finds causes and next steps from timing reports and the PMU	**Weak against traps of the environment** (shared-folder timestamps, macOS mounts)
Reads public specifications (PDF) and implements them	Long conversations get compacted: keeping documents and notes is a must
Keeps working through the night and leaves records	What to build and where to stop is decided by the human
Commits and pushes to GitHub on its own → the history is always kept	
Builds the software environment too: the Eclipse projects, a Debian root file system, procedures for others	

Most of the weaknesses are covered by the human checking on the board and giving the AI the raw numbers.

6.2 What the human does, and how to work well

- **State the goal and the constraints clearly**: "only the CPU is home-made; peripherals and Linux are reused".
- **When in doubt, have options proposed and compared, then choose**: the base, the bus width, the cache configuration.
- **Have it measure before building**: the memory waits were counted before deciding what to work on first.
- **Check both in simulation and on the board**: fill every disagreement as a hole in verification.
- **Have it report in numbers, and look at the raw logs**.
- **Have decisions and lessons kept in documents and notes**: the next session continues from them.
- **Divide the work into themes and commit at clean points**.
- **Give it primary sources**: the RISC-V specification PDFs, the board's logs and reports.

> **Act like the technical manager of a real development team**
>
> Without checking every detail, interview the engineer in charge (Claude Code) about how it thinks and how it intends to check and verify its work, and confirm the quality of the design that way. Even without reading the RTL line by line, the quality becomes visible by asking how it was thought out and how it was checked.

7. Summary and next steps

7.1 Summary

- Claude Code designed an RV64GC CPU, and **Linux booted on it on the Arty A7-100T in 12 days**.
- **Rev.01** in 30 days: L2, pipelined FPU, Zbc / Zicboz, 3.0 CoreMark/MHz, WNS +0.664 ns, 120-minute stress test passed.
- **Rev.02**, the final version, in 38 days: ITLB 16 / DTLB 32, decision 82, **3.034 CoreMark/MHz**, WNS +0.140 ns, stress test passed; **bare-metal development in Eclipse** over JTAG / cJTAG, and **Debian 13 with gcc on the board**.
- The human's role is **the goals and decisions, the board, and checking what the AI reports**.

7.2 Where mmRISC-2 stops, and what is left (ROADMAP)

mmRISC-2 closes with Rev.02 (tag Rev.02 on main). The rule set when the gains began to level off: drop any theme whose estimated upper bound is under a few percent, and close when what is left gives 1 to 2 %. The three themes planned after Rev.01 ended this way:

- **Larger TLBs** (done): page table walks went from 10 to 13 % of user-mode loads to under 0.3 %, so the second-level TLB was not needed.
- **Write-validate** (W1, built, not taken): +0.1 %. What it revealed is left for the future: the core waits for the answer of every store that misses (M1b).
- **Posted uncached stores** (W2, not built): an uncached store costs 13 cycles; under 0.1 % of tftp.

Also left in the ROADMAP: the SD card's DMA writes in the L2, software prefetch, the DTLB check a cycle later for timing, and the larger themes below.

7.3 Next concept: mmRISC-3 (tentative)

A paper study (no design yet) of whether a **dual-issue, out-of-order RV64GC** core can run Linux on the same Arty A7-100T.

Outlook

- **A design of reallocating resources**: slices are already at 93 %. Shrink the L2 and move logic from LUTs to BRAM, LUTRAM and DSPs.
- For scale: NaxRiscv RV64, an out-of-order core for FPGAs, is about 18k LUTs for the core (without FPU and Zb extensions).
- **Stay at 50 MHz** and save resources with fewer stages.
- The biggest fork is **the FPU** (its own renaming / shared with the integer side / reduced).

What mmRISC-2 has shown

- A missed load's value is used 1 or 2 instructions later → **a D$ and L2 that handle several misses at once are a prerequisite** (today the bus takes one read at a time).
- The crux for Linux is **not speculating side effects**: MMIO loads, CSRs, AMOs, exceptions, debug and triggers at commit.
- Expectation: about 1.6 times at best on CoreMark, 1.2 to 1.4 times on kernel loads.

The plan: measure the breakdown of resources → measure NaxRiscv RV64 on the Arty → boot Linux on a 1-wide out-of-order core → go 2-wide → speculative loads and multiple outstanding misses → FPU and predictor. Verification keeps the reference models, bug injection and SIM_BIOS of mmRISC-2, and adds lock-step comparison with Spike at commit from the start. If it does not fit, the fallback is a dual-issue in-order core, or stopping at 1-wide out-of-order.

Appendix A. About the author

Munetomo Maruyama (professional name; legal name: Takanaga Yamazaki), from Sakyo-ku, Kyoto

Joined Hitachi, Ltd. in 1986 and worked on the development of SH microcontrollers. Since then, at Renesas Technology, a venture company and a major semiconductor maker, he has led the development of SoCs for image processing, various microcontrollers and related semiconductor devices, and he still designs microcontroller products today. He has been playing with microcomputers, FPGAs and GPUs since 1978, and has written magazine articles (about 100) and books on microcontrollers since 2000.

Selected magazine articles (in Japanese)

1. A case study of developing an ARM-based system LSI, Design Wave Magazine, May 2006, CQ Publishing
2. Building a general-purpose JTAG debugger for ARM processors, Design Wave Magazine, June 2008, CQ Publishing
3. A thorough study of the parallel processor xCORE, Interface, Nov. 2014 to June 2015 (series), CQ Publishing
4. Driving the massively parallel GPU at full speed directly from C, Transistor Gijutsu, Sept. 2019 (feature), CQ Publishing
5. I built the world's first microprocessor, the 4004, I/O, Nov. 2025, Kohgakusha

Books (in Japanese)

1. H8 Microcontroller Board Ready to Use (2010; expanded edition 2011), CQ Publishing
2. Two Small ARM Microcontroller Boards (2011), CQ Publishing
3. My Special Microcontroller with ARM PSoC, with a board (2013), CQ Publishing
4. My Special Microcontroller with ARM PSoC, development (2013), CQ Publishing
5. Collected Articles on Using SH Microcontrollers (2014), CQ Publishing
6. FPGA Electronics Super Kit (2016), CQ Publishing
7. Learning FPGAs and Computers with the MAX10 Experiment Kit (2016), CQ Publishing
8. The Complete FPGA Electronics All-in-One Kit (2016), CQ Publishing

Appendix B. References

- mmRISC-2 (this project, Apache-2.0): https://github.com/munetomo-maruyama/mmRISC-2 — specifications (CPU_CORE_SPEC, CPU_CACHE_SPEC, CPU_L2_SPEC, CPU_DBG_SPEC), records from the board (BRINGUP, TIMING, BENCH), ROADMAP

- Claude Code: https://claude.com/claude-code
- LiteX (SoC builder) and LiteDRAM, LiteEth, LiteSDCard: https://github.com/enjoy-digital/litex, https://github.com/litex-hub
- The RISC-V Instruction Set Manual, Volume I / II (20260120), The RISC-V Debug Specification 1.0: https://github.com/riscv
- OpenSBI: https://github.com/riscv-software-src/opensbi, riscv-tests: https://github.com/riscv-software-src/riscv-tests
- Arty A7 Reference Manual: https://digilent.com/reference/programmable-logic/arty-a7/start
- NaxRiscv: https://github.com/SpinalHDL/NaxRiscv
- Eclipse Embedded CDT: https://eclipse-embed-cdt.github.io, xPack RISC-V GCC / OpenOCD: https://xpack-dev-tools.github.io
- Debian riscv64: https://wiki.debian.org/RISC-V, mmdebstrap: https://gitlab.mister-muffin.de/josch/mmdebstrap

The figures in this paper are those of mmRISC-2 Rev.02 as of 2026-10-20. The text and figures of this paper were also written by Claude Code under the author's direction. A Japanese version is in Doc/20261020_WhitePaper_mmRISC-2_J.pdf.